

Лабораторна робота №3

Мета роботи

Ознайомлення з принципами онлайн-обробки транзакцій (OLTP). Набуття практичних навичок маніпулювання даними в реляційній базі даних PostgreSQL за допомогою мови DML (Data Manipulation Language). Практичне застосування операторів SELECT, INSERT, UPDATE та DELETE із суворим використанням фільтрації WHERE для безпечної зміни записів.

1. Отримання даних (SELECT)

Для імітації роботи адміністратора пункту прокату було виконано запити на вибірку конкретних даних із застосуванням фільтрації.

Запит 1.1: Виведення списку клієнтів із певною поштою

- **Мета:** Отримати імена, прізвища та електронні адреси лише тих клієнтів, які користуються поштою на домені kpi.ua.
- **Очікуваний результат:** Таблиця з двома клієнтами (Олександр та Артем), без виведення зайвих колонок (наприклад, customer_id).
- **SQL-код:**

```
SELECT first_name, last_name, email
```

```
FROM CUSTOMER
```

```
WHERE email LIKE '%@kpi.ua';
```

- **Результат виконання:** Запит виконано успішно.

	first_name character varying (100) 🔒	last_name character varying (100) 🔒	email character varying (150) 🔒
1	Oleksandr	Kovalenko	alex.koval@kpi.ua
2	Artem	Tkachenko	artem.tkach@kpi.ua

Запит 1.2: Пошук бюджетних фільмів

- **Мета:** Знайти назви фільмів та їх вартість, де ціна оренди менша за 5 доларів.
- **SQL-код:**

```
SELECT title, rental_rate
```

```
FROM FILM
```

```
WHERE rental_rate < 5.00;
```

- **Результат виконання:** Запит виконано успішно (знайдено «The Matrix», «The Hangover», «Pulp Fiction»).

	title character varying (255)	rental_rate numeric (5,2)
1	The Matrix	4.99
2	The Hangover	3.50
3	Pulp Fiction	2.99

2. Додавання нових даних (INSERT)

Було імітовано бізнес-процес реєстрації нового клієнта в системі та додавання нової категорії фільмів.

Запит 2.1: Реєстрація нового клієнта

- **Мета:** Додати до таблиці CUSTOMER новий запис із дотриманням усіх обмежень цілісності (наявність @ в email).
- **Очікуваний результат:** Успішне створення запису з автоматичним присвоєнням customer_id завдяки типу SERIAL.
- **SQL-код:**

```
INSERT INTO CUSTOMER (first_name, last_name, email)
```

```
VALUES ('Taras', 'Shevchenko', 'taras.kobzar@gmail.com');
```

```
SELECT * FROM CUSTOMER WHERE last_name = 'Shevchenko';
```

- **Результат виконання:** Запит виконано успішно. Новий рядок з'явився у таблиці.

	customer_id [PK] integer	first_name character varying (100)	last_name character varying (100)	email character varying (150)
1	2	Mariia	Shevchenko	masha.sheva@gmail.c...
2	6	Taras	Shevchenko	taras.kobzar@gmail.com

Запит 2.2: Додавання тестової категорії

- **Мета:** Створити нову категорію жанру.
- **SQL-код:**

```
INSERT INTO CATEGORY (name) VALUES ('Documentary_Test');
```

```
INSERT 0 1

Query returned successfully in 65 msec.
```

3. Зміна існуючих даних (UPDATE)

Було перевірено функціонал безпечного оновлення записів із обов'язковим використанням умови WHERE, щоб уникнути масового перезапису даних.

Запит 3.1: Оновлення електронної пошти клієнта

- **Мета:** Змінити пошту щойно зареєстрованого клієнта "Taras Shevchenko".
- **Очікуваний результат:** Зміна значення в колонці email лише для одного конкретного рядка.
- **SQL-код:**

```
UPDATE CUSTOMER
```

```
SET email = 'taras.new@kpi.ua'
```

```
WHERE first_name = 'Taras' AND last_name = 'Shevchenko';
```

```
SELECT * FROM CUSTOMER WHERE first_name = 'Taras';
```

- **Результат виконання:** Запит виконано успішно. Значення змінено безпечно.

	customer_id [PK] integer	first_name character varying (100)	last_name character varying (100)	email character varying (150)
1	6	Taras	Shevchenko	taras.new@kpi.ua

Запит 3.2: Зміна стану фізичного диска (інвентарю)

- **Мета:** Зімітувати пошкодження диска під час прокату шляхом зміни значення нашого кастомного типу disk_condition.
- **SQL-код:**

```
UPDATE INVENTORY
```

```
SET condition = 'Damaged'
```

```
WHERE inventory_id = 3;
```

- **Результат виконання:** Запит виконано успішно. Обмеження типу ENUM пропустило значення 'Damaged'.

```
UPDATE 1
Query returned successfully in 58 msec.
```

	inventory_id [PK] integer	film_id integer	condition disk_condition
1	3	2	Damaged

4. Безпечне видалення даних (DELETE)

Операція видалення є критичною в OLTP-системах, тому перевірялась обережність застосування фільтрів.

Запит 4.1: Видалення тестової категорії

- **Мета:** Видалити жанр 'Documentary_Test', який був створений помилково на етапі INSERT.

- **Очікуваний результат:** Видалення рівно одного рядка з таблиці CATEGORY.
- **SQL-код:**

```
SELECT * FROM CATEGORY WHERE name = 'Documentary_Test';
```

```
DELETE FROM CATEGORY
```

```
WHERE name = 'Documentary_Test';
```

```
SELECT * FROM CATEGORY WHERE name = 'Documentary_Test';
```

- **Результат виконання:** Запит виконано успішно. Запис було видалено, що підтверджено порожнім результатом останнього SELECT.

category_id	name
[PK] integer	character varying (100)

Запит 4.2: Видалення тестового клієнта

- **Мета:** Видалити клієнта "Taras Shevchenko" з бази. Оскільки він ще не брав фільмів в оренду (немає записів у таблиці RENTAL), зовнішні ключі (Foreign Keys) дозволять безпечно його видалити.
- **SQL-код:**

```
DELETE FROM CUSTOMER
```

```
WHERE email = 'taras.new@kpi.ua';
```

- **Результат виконання:** Запит виконано успішно.

```
DELETE 1
Query returned successfully in 58 msec.
```

Висновок

Під час виконання лабораторної роботи №3 було успішно відпрацьовано принципи маніпулювання даними (DML) у середовищі PostgreSQL. Налаштована раніше реляційна схема безперебійно обробляє OLTP-транзакції: реєстрацію нових записів (INSERT), цільову вибірку за критеріями (SELECT), модифікацію існуючої інформації (UPDATE) та контрольоване видалення (DELETE).

Усі операції зміни та видалення проводилися з суворим дотриманням найкращих практик безпеки — із застосуванням оператора WHERE, що унеможливило випадкове спотворення або втрату критично важливих даних (наприклад, очищення всієї таблиці клієнтів). Перевірочні запити підтвердили абсолютну коректність та ефективність виконаних SQL-інструкцій.